Case Study on the Application of Bayes' Theorem in Spam Detection

[Student Name]

[Department of Computer Science, University Name]

[Course Code]: Basics of Neural Networks and Deep Learning

[Instructor Name]

November 9, 2026

Case Study on the Application of Bayes' Theorem in Spam Detection

Spam, or unsolicited bulk email, wastes bandwidth and user attention, and it is a common way to deliver phishing links and malware (Dada et al., 2019). Spammers keep changing their wording, so hand-written rules go out of date quickly, and filters that learn from labeled examples have become the standard defence (Blanzieri & Bryl, 2008). The naive Bayes classifier is one of the oldest of these methods and is still widely used because it is fast and works well on text (Guzella & Caminhas, 2009).

Overview of Bayes' Theorem

Bayes' theorem updates the probability of a hypothesis H after observing evidence E:

$$P(H \mid E) = P(E \mid H)\, P(H) / P(E) \qquad (1)$$

Here $P(H)$ is the prior, $P(E \mid H)$ is the likelihood, $P(E)$ is the overall probability of the evidence, and $P(H \mid E)$ is the posterior. In spam filtering, the hypothesis is that a message belongs to class $c \in \{\text{spam, ham}\}$, where "ham" means legitimate mail, and the evidence is the words in the message. The theorem turns a hard question ("given these words, is this spam?") into quantities that can be counted in labeled data. For example, in the training data used below, 31.4% of messages are spam, and the word "click" appears in 54.4% of spam and 4.7% of ham. Therefore $P(\text{spam} \mid \text{click}) = (0.544 \times 0.314) / (0.544 \times 0.314 + 0.047 \times 0.686) \approx 0.84$. A real filter combines such evidence from every word.

Description of the Dataset

A Bayesian filter learns from emails that have each been labeled spam or ham. Common public corpora include Ling-Spam, Enron-Spam, and the SpamAssassin public corpus (Dada et al., 2019). This study uses the SpamAssassin corpus: 6,046 raw emails, of which 4,150 are ham (68.6%) and 1,896 are spam (31.4%). The ham set includes 250 "hard" messages, such as

newsletters, that resemble spam. This kind of data is imbalanced, so the prior is not 0.5. It is also sparse: the vocabulary has tens of thousands of words, but each message uses only a few hundred. And it changes over time as spam campaigns change (Blanzieri & Bryl, 2008). The corpus was split into 4,836 training messages (80%) and 1,210 test messages (20%). Both parts kept the original class ratio.

Data Preprocessing

Parsing and Tokenization

Each email was parsed with a MIME parser. The subject and text parts were kept, HTML tags were removed, and the text was lowercased. URLs, email addresses, and numbers were replaced with placeholder tokens (URLTOKEN, EMAILTOKEN, NUMTOKEN). Tokenization then split the text into words of two or more letters. Stop words such as "the" were removed because they carry no class evidence.

Vectorization

Each message was then turned into a numeric vector using the bag-of-words model. A vocabulary of $V = 23{,}164$ words that appear in at least two training messages was built. Each message became a vector $x = (x_1, \ldots, x_v)$, where x_i counts how often word i occurs. Two alternatives were also tested. Binary presence/absence features are used by Bernoulli naive Bayes. TF-IDF weighting gives less weight to words that appear in many documents (Salton & Buckley, 1988).

How the Theorem Classifies Messages

By Bayes' theorem, $P(c \mid x) \propto P(c)\, P(x \mid c)$. Estimating $P(x \mid c)$ for every possible combination of words is impossible. The *naive* assumption solves this by treating words as independent once the class is known, which gives the multinomial naive Bayes decision rule:

$$\hat{c} = \operatorname{argmax}_{c} [\, \log P(c) + \Sigma_i \, x_i \log P(w_i \mid c) \,] \qquad (2)$$

Training only requires counting. $P(c)$ is the share of training messages in class c. Each word likelihood uses Laplace smoothing: $P(w_i \mid c) = (N_{ci} + 1) / (N_c + V)$, where N_{ci} is how often word i occurs in class c and N_c is the total word count of that class. Without smoothing, a word never seen in training ham would get probability zero and cancel all other evidence. Logarithms are added instead of multiplying raw probabilities, which would underflow (Guzella & Caminhas, 2009). To classify a new email, the filter computes Equation 2 for both classes, or equivalently computes $P(\text{spam} \mid x)$, and marks the email as spam if that probability is above a threshold (0.5 by default). In effect, each word votes with weight $\log[P(w \mid \text{spam}) / P(w \mid \text{ham})]$. In the trained model, the strongest spam words included “mortgage” and “annuity,” and the strongest ham words included “kernel” and “perl.”

Performance Analysis

The classifiers were built with scikit-learn and tested on the 1,210 held-out messages, with spam as the positive class. Accuracy alone can be misleading when the classes are imbalanced, so the evaluation also reports precision (the share of flagged messages that really are spam), recall (the share of spam that was caught), F1, specificity, and AUC (Dada et al., 2019).

Table 1

Test-Set Performance of Three Naive Bayes Variants (n = 1,210)

Model	Accuracy	Precision	Recall	F1	Specificity	AUC	5-fold CV
Multinomial (counts)	.962	.956	.921	.938	.981	.990	.970
Bernoulli (binary)	.948	.968	.863	.912	.987	.993	.947
Multinomial (TF-IDF)	.975	.976	.945	.960	.989	.998	.974

Note. Model names give the naive Bayes event model and features; CV = cross-validation; AUC = area under the ROC curve. Smoothing $\alpha = 1$ for count models, $\alpha = 0.1$ for TF-IDF.

The standard word-count model reached 96.2% accuracy, with 349 true positives, 30 false negatives (spam that reached the inbox), 16 false positives (ham marked as spam), and 815

true negatives. Five-fold cross-validation gave 97.0% ± 0.3%. TF-IDF weighting improved every metric (F1 = .960, only nine false positives), because it stops common but uninformative words from dominating Equation 2. The Bernoulli model had high precision but the lowest recall. It ignores how often a word is repeated, which shows that the choice of event model matters (Guzella & Caminhas, 2009).

The two kinds of error do not cost the same, because losing a real email is worse than receiving a spam message. Raising the decision threshold from 0.5 to 0.99 cut false positives from 16 to 11, but recall fell from .921 to .900. AUC values above .99 show that spam and ham are well separated, so most errors come from where the threshold sits. Naive Bayes is fast and interpretable, but the independence assumption ignores word order, its probability estimates tend to be overconfident, and spammers can evade it by misspelling words or adding ordinary ham words (Blanzieri & Bryl, 2008). Even so, it remains a strong baseline and a good first stage in a layered filter.

Conclusion

Bayes' theorem gives spam filtering a simple principle: combine how common spam is with how strongly each word points to it. Once the emails are tokenized and vectorized, a naive Bayes model can be trained just by counting. On the SpamAssassin corpus it caught 92%–95% of spam and flagged only 1%–2% of legitimate mail, with TF-IDF giving the best results.

References

Blanzieri, E., & Bryl, A. (2008). A survey of learning-based techniques of email spam filtering. *Artificial Intelligence Review, 29*(1), 63–92. https://doi.org/10.1007/s10462-009-9109-6

Dada, E. G., Bassi, J. S., Chiroma, H., Abdulhamid, S. M., Adetunmbi, A. O., & Ajibuwa, O. E. (2019). Machine learning for email spam filtering: Review, approaches and open research problems. *Heliyon, 5*(6), Article e01802. https://doi.org/10.1016/j.heliyon.2019.e01802

Guzella, T. S., & Caminhas, W. M. (2009). A review of machine learning approaches to spam filtering. *Expert Systems with Applications, 36*(7), 10206–10222. https://doi.org/10.1016/j.eswa.2009.02.037

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0